

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

CultureQuest - Platforma mobilă de explorare urbană bazată pe proximitate, cu recomandări personalizate prin învățare federată

Andrei Cozma

Coordonator științific:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCUREȘTI

2026

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA OF BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

CultureQuest - Proximity-Based Mobile Platform for Urban
Exploration with Personalized Recommendations via Federated
Learning

Andrei Cozma

Thesis advisor:

Prof. dr. ing. Radu-Ioan Ciobanu

BUCHAREST

2026

CUPRINS

1	Introducere	1
1.1	Context	1
1.2	Definirea problemei	1
1.3	Obiective	2
1.4	Soluția propusă	2
1.5	Rezultatele obținute	3
1.6	Structura lucrării	3
2	Analiza Cerințelor	4
2.1	Utilizatori țintă și cazuri de utilizare	4
2.2	Cerințe funcționale	4
2.3	Cerințe non-funcționale	5
2.4	Domeniul de aplicare al proiectului	6
3	Soluții Existente	7
3.1	Aplicații existente pentru explorare urbană	7
3.2	Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări	7
3.3	Învățarea federată: Stadiul actual	7
3.4	Alternative respinse	7
3.5	Tehnologiile utilizate și justificarea acestora	7
3.5.1	Flutter și Riverpod	7
3.5.2	FastAPI, MongoDB și Redis	7
3.5.3	Motorul de Rutare OSRM	7
3.5.4	Perceptron multi-strat	7
4	Soluția Propusă	8
4.1	Prezentare generală a sistemului	8

4.2	Arhitectura aplicației mobile	8
4.3	Arhitectura serviciilor <i>backend</i>	8
4.4	Proiectarea sistemului de învățare federată	8
4.4.1	Arhitectura modelului	8
4.4.2	Antrenarea locală	8
4.4.3	Agregare asincronă și reducerea în funcție de vechime	8
4.4.4	Limitarea gradientilor	8
4.4.5	Confidențialitate diferențială	8
4.4.6	Controlul concurenței	8
4.5	Fluxul de generare a rutelor	8
4.6	Proiectarea elementelor de gamificare	8
4.7	Hartă și navigare	8
4.8	Arhitectura de confidențialitate	8
5	Detalii de Implementare	9
5.1	Implementarea aplicației mobile	9
5.1.1	Serviciul client de învățare federată	9
5.1.2	Adăugarea de zgomot pentru asigurarea confidențialității diferențiale	9
5.1.3	Bufferul local de interacțiuni și persistența datelor	9
5.1.4	Stocarea temporară a obiectivelor pentru funcționarea offline	9
5.1.5	Algoritmul de generare a rutelor	9
5.1.6	Sistemul de misiuni și tehnologia de <i>geofencing</i>	9
5.2	Implementarea serviciilor <i>backend</i>	9
5.2.1	Serviciul de agregare	9
5.2.2	Punctele de acces API	9
5.2.3	Sistemul de recenzii și moderarea conținutului	9
5.3	Fluxul etapei de pre-antrenare	9
5.4	<i>Framework-ul</i> pentru simularea procesului de învățare federată	9

5.5	Dificultăți întâmpinate în procesul de implementare	9
6	Evaluare	10
6.1	Corectitudinea funcțională	10
6.2	Performanța modelului de învățare federată	10
6.2.1	Rezultatele etapei de pre-antrenare	10
6.2.2	Rezultatele simulării procesului de învățare federată	10
6.3	Evaluarea confidențialității și robusteții	10
6.4	Analiză comparativă și discuții	10
7	Concluzii	11
7.1	Concluzii generale	11
7.2	Direcții de cercetare și lucrări viitoare	11
	Bibliografie	12
	Anexe	13
	Anexa A Extrase de cod	14

SINOPSIS

CultureQuest este o platformă mobilă de explorare urbană bazată pe proximitate, oferind recomandări personalizate legate de obiectivele turistice și recreative dintr-un oraș, fără ca datele utilizatorului să părăsească dispozitivul. Problema pe care o tratez este că aplicațiile de tip *city-guide* oferă aceleași sugestii tuturor, indiferent de interese, pentru că personalizarea ar presupune salvarea centralizată a istoricului de interacțiuni cu obiectivele sau monitorizarea traseului, ceea ce ridică probleme din punctul de vedere al confidențialității. Pentru a evita acest compromis, am implementat o aplicație Flutter în care datele sunt salvate doar local, interacțiunile vizibile altor utilizatori precum recenziile sunt anonime și adăugate strict de vizitatori, iar rutele turistice generate sunt individualizate prin intermediul unei rețele neurale de tip perceptron multi-strat (MLP, *Multi-Layer Perceptron*). Modelul global inițial este pre-antrenat pe seturi de date publice, iar acesta este modificat și îmbunătățit prin tehnici de învățare federată (FL, *Federated Learning*) rulate asincron pe dispozitivele clienților. Recomandările de rute combină scorul oferit de model pentru o atracție turistică cu proximitatea acesteia, iar un sistem de misiuni gamificate încurajează explorarea și colectarea de noi date. Printr-o simulare care include mai multe acțiuni - mai exact 600 - se confirmă funcționarea fluxului FL în mod corespunzător.

ABSTRACT

CultureQuest is a proximity-based mobile urban exploration platform, offering personalized recommendations for tourist and recreational attractions in a city, without the user's data leaving the device. The problem I address is that city-guide type applications offer the same suggestions to everyone regardless of interests, because personalization would require centrally storing a user's interaction history with these attractions or tracking their route, both of which raise privacy concerns. To avoid this trade-off, I developed a Flutter application where data is saved only locally, interactions visible to other users such as reviews are anonymous and can only be added by actual visitors and the generated tourist routes are personalized through a multi-layer perceptron (MLP) neural network. The initial global model is pre-trained on public datasets, and it is modified and improved through federated learning (FL) techniques run asynchronously on the client devices. Route recommendations combine the model's score for a tourist attraction with its proximity, while a gamified quest system encourages exploration and data collection. A simulation that includes multiple client actions - 600, to be exact - confirms that the FL flow is working properly.

1 INTRODUCERE

Acest capitol prezintă contextul în care se situează proiectul și problema care îl motivează (1.1, 1.2), obiectivele propuse (1.3), o privire de ansamblu asupra soluției dezvoltate (1.4) și a rezultatelor obținute (1.5), precum și modul în care este organizată restul lucrării (1.6).

1.1 Context

Telefoanele mobile au devenit, în ultimii ani, principalul instrument prin care utilizatorii interacționează cu mediul din jurul lor, iar aplicațiile mobile bazate pe locație - de la Google Maps, TripAdvisor și GetYourGuide, la diverse aplicații de tip *city-guide* - au devenit principala modalitate prin care turiștii și localnicii descoperă obiective turistice și culturale dintr-un oraș și primesc recomandări din ce în ce mai adaptate propriilor interese [3].

Pentru a oferi recomandări personalizate, aceste aplicații se bazează pe colectarea și procesarea centralizată a istoricului de interacțiuni al utilizatorului - locurile vizitate, recenziile adăugate, timpul petrecut în diverse locații - lucru care ridică probleme de confidențialitate, întrucât utilizatorul pierde controlul asupra modului în care îi sunt folosite și stocate datele legate de comportament.

Învățarea federată (FL, *Federated Learning*) apare ca alternativă la acest model centralizat: modelul este antrenat local, pe dispozitivul utilizatorului, folosind datele acestuia, iar către server sunt trimise doar actualizările rezultate, nu datele brute, pentru a fi agregate într-un model global. FL este deja folosită la scară destul de mare în sisteme de producție [2], ceea ce motivează aplicarea ei și în cadrul acestei platforme, pentru a oferi recomandări de rute turistice fără a compromite confidențialitatea clienților.

1.2 Definirea problemei

Majoritatea aplicațiilor de explorare urbană dezvoltate oferă, în absența unui profil construit din acțiunile turiștilor pe platformă, aceleași rute tuturor, indiferent de interesele fiecăruia, chiar dacă acestea sunt legate de artă, gastronomie, natură sau alte evenimente culturale. Această abordare generică este mai simplă de dezvoltat și de întreținut, însă nu ține cont de faptul că două persoane care vizitează același oraș pot avea preferințe complet diferite în ceea ce privește modul în care doresc să îl exploreze.

Varianta prin care o aplicație devine personalizată presupune, de regulă, ca istoricul de interacțiuni al utilizatorului să fie încărcat și procesat pe un server central, exact tipul de practică care aduce în discuție problemele de confidențialitate menționate mai sus. Utiliza-

torului i se prezintă, astfel, un compromis: fie acceptă recomandări generice, fie acceptă ca datele sale comportamentale să fie colectate central pentru a primi recomandări adaptate intereselor sale - iar acest compromis este punctul de plecare al proiectului de față.

1.3 Obiective

Pornind de la acest compromis, proiectul de față își propune să arate că personalizarea și păstrarea confidențialității nu se exclud reciproc, prin trei obiective principale:

- (a) O aplicație mobilă prin care utilizatorul poate descoperi obiective culturale dintr-un oraș printr-o explorare bazată pe proximitate, cu un sistem de *quest-uri* și puncte obținute pentru interacțiunile pe platformă, care îl încurajează să viziteze locuri noi.
- (b) Personalizarea recomandărilor de rute pentru fiecare utilizator, fără ca datele acestuia - istoricul de interacțiuni, locațiile vizitate - să părăsească dispozitivul.
- (c) Implementarea unui flux complet de FL: pre-antrenarea modelului global, ajustarea fină (*fine-tuning*), antrenarea locală prin FedAvg [4] și agregarea asincronă a actualizărilor primite de la clienți, cu garanții de confidențialitate.

1.4 Soluția propusă

Soluția dezvoltată este o aplicație mobilă realizată în Flutter, prin care utilizatorul poate vedea pe hartă obiectivele turistice și evenimentele din apropiere. Pe lângă explorare, aplicația oferă un sistem de *quest-uri*, care încurajează vizitarea unor obiective noi, dar și posibilitatea de a lăsa recenzii, povești sau chiar alte *quest-uri* pentru locurile vizitate - tot ce adaugă utilizatorii devine, în timp, parte din imaginea pe care comunitatea o are despre fiecare atracție.

Pentru personalizare, fiecare utilizator are pe dispozitiv un model mic, un perceptron multi-strat (MLP, *Multi-Layer Perceptron*) cu un strat de intrare de 22 de caracteristici, două straturi ascunse (32 și 16 neuroni) și un neuron de ieșire, care reprezintă un scor de recomandare pentru fiecare obiectiv. Pe baza acestui scor, fiecărui obiectiv i se asociază o etichetă care arată cât de recomandat este pentru utilizator, în funcție de interesele acestuia, de ora curentă, de tipul locului etc., iar la generarea rutei, scorul fiecărui obiectiv este combinat cu distanța față de poziția curentă. Modelul global inițial este pre-antrenat cu ajutorul unor seturi de date din mediul online, apoi antrenat pe dispozitiv prin FL, iar *backend-ul* (FastAPI, MongoDB, Redis) coordonează învățarea asincronă.

1.5 Rezultatele obținute

La nivel general, rezultatul este o aplicație Flutter funcțională, care reunește harta cu obiective și evenimente din apropiere, generarea de rute personalizate și sistemul de *quest-uri*. Personalizarea descrisă în secțiunea anterioară este vizibilă direct în aplicație, pentru fiecare obiectiv din apropiere și în cadrul rutelor generate.

Pe partea de FL, modelul de pe dispozitiv pornește de la o variantă a modelului global trecută și prin etapa de *fine-tuning*, iar antrenarea locală se face prin FedAvg. Agregarea pe server este asincronă, cu o reducere în funcție de vechimea (*staleness discount*) actualizărilor venite de la clienți [5] - o metodă prin care influența lor asupra modelului global este diminuată. O simulare de 600 de runde confirmă funcționarea în parametri buni a fluxului FL.

1.6 Structura lucrării

Capitolul 1 a introdus, la nivel general, contextul, problema, obiectivele, soluția propusă și rezultatele obținute. Capitolele următoare intră în detaliu pe fiecare dintre aceste aspecte.

Capitolul 2 stabilește cerințele aplicației, pornind de la cazurile de utilizare ale unui turist sau localnic care explorează un oraș, și le împarte în cerințe funcționale și non-funcționale. Tot aici se delimitează și domeniul de aplicare al proiectului.

Capitolul 3 trece în revistă aplicațiile de explorare urbană existente și modul în care acestea personalizează recomandările, comparând abordările centralizate cu FL. Se poziționează CultureQuest pe baza taxonomiei FL, se explică de ce alte alternative de algoritmi FL nu se potrivesc *design-ului* asincron al platformei și se justifică tehnologiile alese.

Capitolul 4 detaliază arhitectura propusă a sistemului. Aici se discută structura aplicației mobile și a serviciilor *backend*, proiectarea modelului MLP și a procesului FL, fluxul de generare a rutelor, elementele de gamificare și arhitectura de confidențialitate.

După proiectare, **Capitolul 5** trece la implementarea efectivă - serviciul client FL și mecanismul de confidențialitate diferențială, serviciile *backend* de agregare și de moderare a conținutului, etapa de pre-antrenare și *framework-ul* folosit pentru simularea procesului FL.

Capitolul 6 evaluează rezultatele obținute - corectitudinea funcțională a aplicației, performanța modelului rezultată din pre-antrenare și din fluxul FL, efectul confidențialității diferențiale și al limitării asupra robusteții - și include o discuție comparativă față de alternativele respinse.

Capitolul 7 încheie lucrarea cu o privire de ansamblu asupra obiectivelor stabilite inițial, în raport cu ce a fost dezvoltat și propune direcții viitoare - de exemplu, personalizarea suplimentară per utilizator sau alte tipuri de agregare în ceea ce privește modelul rețelei neurale.

2 ANALIZA CERINȚELOR

Capitolul de față detaliază cerințele care reies din obiectivele stabilite la 1.3: cine sunt utilizatorii și ce cazuri de utilizare trebuie acoperite (2.1), ce funcționalități rezultă de aici (2.2), ce constrângeri non-funcționale se impun (2.3) și ce rămâne în afara domeniului de aplicare (2.4). Toate aceste cerințe stau la baza deciziilor de proiectare din Capitolul 4, fiind verificate ulterior în evaluarea din Capitolul 6.

2.1 Utilizatori țintă și cazuri de utilizare

Utilizatorul principal al aplicației este un turist sau un localnic care explorează un oraș - sau o zonă a acestuia - cu dorința de a vizita obiective turistice sau culturale din proximitate, adaptate intereselor proprii. Primul caz de utilizare este cel de generare de rute și suport pe parcursul urmăririi acestora, în care utilizatorul vede pe hartă poziția sa curentă, locurile de interes din apropiere și primește o rută care combină preferințele sale cu acestea. Mai mult, interacțiunile cu aplicația - vizite, misiuni (*quest-uri*) finalizate, recenzii - determină ajustarea, prin FL, a modelului de pe dispozitiv - și, implicit, și a celui global - fără ca istoricul brut al interacțiunilor să ajungă pe server. Un al doilea caz de utilizare este cel de contribuție de conținut, în care utilizatorul poate propune obiective noi, povești, misiuni sau recenzii, toate acestea necesitând aprobarea de către alte persoane care folosesc aplicația sau de către un administrator. Prin conturile de tip administrator, relevante mai ales pentru 2.2, se aprobă sau se respinge conținutul propus de utilizatorii obișnuiți.

2.2 Cerințe funcționale

Pornind de la cazurile de utilizare din secțiunea anterioară, cerințele funcționale au fost sintetizate în tabelul 1, fiecare cu o descriere și secțiunile din Capitolele 4-6 unde cerința este tratată. Primul grup (de la **FR-1** la **FR-4**) sumarizează bucla principală de explorare: afișarea pe hartă a obiectivelor turistice din proximitate, generarea rutei care combină scorul modelului de tip MLP cu distanța față de poziția curentă, *quest-urile* disponibile la obiectivele vizitate și posibilitatea de a adăuga sau vizualiza recenzii.

Al doilea grup (de la **FR-5** la **FR-7**) ține de personalizare și administrare: **FR-5** descrie fluxul procesului FL - primirea parametrilor modelului actualizat de la server, urmată de antrenarea locală, fără ca istoricul interacțiunilor să ajungă pe server, **FR-6** descrie controalele de confidențialitate - asigurarea confidențialității diferențiale (DP, *Differential Privacy*), vizibilitatea asupra datelor, iar **FR-7** descrie rolul de administrator - aprobarea sau respingerea conținutului propus.

Tabelul 1: Cerințe funcționale

ID	Descriere	Secțiune
FR-1	Obiective din proximitate pe hartă.	4.7, 5.1.4, 6.1
FR-2	Rute personalizate: scor model MLP și proximitate.	4.5, 5.1.5, 6.1
FR-3	<i>Quest-uri</i> la obiectivele vizitate; puncte și progres.	4.6, 5.1.6, 6.1
FR-4	Adăugare și vizualizare recenzii pentru obiective.	5.2.3, 6.1
FR-5	Sincronizare model și antrenare locală, fără date colectate pe server.	4.4, 5.1.1–5.1.3, 5.2.1, 6.2
FR-6	Zgomot DP aplicat pe actualizări; vizibilitate asupra datelor trimise.	4.8, 5.1.2, 6.3
FR-7	Administrator: aprobă sau respinge conținut propus (obiective, povești, <i>quest-uri</i> , recenzii).	5.2.3, 6.1

2.3 Cerințe non-funcționale

La fel ca în secțiunea 2.2, cerințele non-funcționale au fost sintetizate în tabelul 2, cu aceeași structură (descriere și secțiunile din Capitolele 4-6 unde sunt tratate). Primul grup (de la **NFR-1** la **NFR-3**) are legătură cu constrângerile impuse pe dispozitiv: **NFR-1** cere ca nicio informație privată despre utilizator - exceptând lista de interese - să nu părăsească telefonul, **NFR-2** cere ca obiectivele turistice să rămână disponibile și fără conexiune la internet, iar **NFR-3** cere ca antrenarea locală să nu fie complexă din punct de vedere computațional pentru un telefon obișnuit - ceea ce a motivat alegerea unui model MLP de dimensiuni reduse.

Al doilea grup ține de partea de *backend*: **NFR-4** impune ca serviciul de agregare să suporte actualizări asincrone primite de la mai mulți clienți - fără conflicte între runde, iar **NFR-5** impune ca utilizatorii să primească recomandări utile încă de la început printr-un model global pre-antrenat [2] - și din caracteristici bazate pe ariile de interes asociate fiecărui obiectiv - până când procesul FL își face efectul.

Tabelul 2: Cerințe non-funcționale

ID	Descriere	Secțiune
NFR-1	Confidențialitate: nicio interacțiune a utilizatorului nu părăsește dispozitivul.	4.8, 5.1.2–5.1.3, 6.3
NFR-2	Reziliență offline: obiective turistice stocate în <i>cache</i> local.	5.1.4, 6.1
NFR-3	Cost redus de antrenare pe dispozitiv.	4.4.1, 5.1.1, 6.2.1
NFR-4	Scalabilitatea <i>backend-ului</i> la actualizările asincrone.	4.4.6, 5.2.1, 6.2.2
NFR-5	<i>Cold start</i> : recomandări utile pentru utilizatori folosind modelul pre-antrenat.	4.4.1, 5.3, 6.2.1

2.4 Domeniul de aplicare al proiectului

Cerințele funcționale și non-funcționale stabilite anterior delimitează domeniul de aplicare al proiectului: aplicația mobilă cu recomandări de rute turistice, pornind de la pre-antrenarea globală și fiind îmbunătățite prin FL. Rămâne în afara domeniului de aplicare personalizarea suplimentară la nivel de utilizator prin straturi separate ale modelului MLP (*personal head*), care nu ar fi trimise către server, rezultând într-o arhitectură de tip FedPer [1] - propusă ca direcție de lucrări viitoare, dar neimplementată în versiunea curentă.

Având stabilite cerințele și limitele proiectului, Capitolul 3 trece în revistă soluțiile existente pentru explorare urbană (Google Maps, TripAdvisor, alte aplicații de tip *city-guide*) și realizează o analiză comparativă între abordările centralizate și cele bazate pe FL pentru recomandări.

3 SOLUȚII EXISTENTE

3.1 Aplicații existente pentru explorare urbană

3.2 Analiză comparativă între abordările centralizate și învățarea federată pentru recomandări

3.3 Învățarea federată: Stadiul actual

3.4 Alternative respinse

3.5 Tehnologiile utilizate și justificarea acestora

3.5.1 Flutter și Riverpod

3.5.2 FastAPI, MongoDB și Redis

3.5.3 Motorul de Rutare OSRM

3.5.4 Perceptron multi-strat

4 SOLUȚIA PROPUȘĂ

4.1 Prezentare generală a sistemului

4.2 Arhitectura aplicației mobile

4.3 Arhitectura serviciilor *backend*

4.4 Proiectarea sistemului de învățare federată

4.4.1 Arhitectura modelului

4.4.2 Antrenarea locală

4.4.3 Agregare asincronă și reducerea în funcție de vechime

4.4.4 Limitarea gradientilor

4.4.5 Confidențialitate diferențială

4.4.6 Controlul concurenței

4.5 Fluxul de generare a rutelor

4.6 Proiectarea elementelor de gamificare

4.7 Hartă și navigare

4.8 Arhitectura de confidențialitate

5 DETALII DE IMPLEMENTARE

5.1 Implementarea aplicației mobile

5.1.1 Serviciul client de învățare federată

5.1.2 Adăugarea de zgomot pentru asigurarea confidențialității diferențiale

5.1.3 Bufferul local de interacțiuni și persistența datelor

5.1.4 Stocarea temporară a obiectivelor pentru funcționarea offline

5.1.5 Algoritmul de generare a rutelor

5.1.6 Sistemul de misiuni și tehnologia de *geofencing*

5.2 Implementarea serviciilor *backend*

5.2.1 Serviciul de agregare

5.2.2 Punctele de acces API

5.2.3 Sistemul de recenzii și moderarea conținutului

5.3 Fluxul etapei de pre-antrenare

5.4 *Framework-ul* pentru simularea procesului de învățare federată

5.5 Dificultăți întâmpinate în procesul de implementare

6 EVALUARE

6.1 Corectitudinea funcțională

6.2 Performanța modelului de învățare federată

6.2.1 Rezultatele etapei de pre-antrenare

6.2.2 Rezultatele simulării procesului de învățare federată

6.3 Evaluarea confidențialității și robusteții

6.4 Analiză comparativă și discuții

7 CONCLUZII

a

7.1 Concluzii generale

a

7.2 Direcții de cercetare și lucrări viitoare

a

BIBLIOGRAFIE

- [1] M. G. Arivazhagan, V. Aggarwal, A. K. Singh, and S. Choudhary. Federated learning with personalization layers. *arXiv preprint arXiv:1912.00818*, 2019.
- [2] K. Bonawitz, H. Eichner, N. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, H. B. McMahan, T. Van Overveldt, D. Petrou, D. Ramage, and J. Roselander. Towards federated learning at scale: System design. In *Proceedings of the 2nd SysML Conference*, 2019.
- [3] D. Gavalas, C. Konstantopoulos, K. Mastakas, and G. Pantziou. Mobile recommender systems in tourism. *Journal of Network and Computer Applications*, 39:319–333, 2014.
- [4] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas. Communication-efficient learning of deep networks from decentralized data. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2017.
- [5] C. Xie, S. Koyejo, and I. Gupta. Asynchronous federated optimization. *arXiv preprint arXiv:1903.03934*, 2019.

ANEXE

Anexele sunt opționale. Ce poate intra în anexe:

- Exemplu de fișier de configurare sau compilare;
- Un tabel mai mare de o jumătate pagină;
- O figura mai mare de o jumătate pagină;
- O secvență de cod sursa mai mare de jumătate pagină;
- Un set de capturi de ecran (“screenshot”-uri);
- Un exemplu de rulare a unor comenzi plus rezultatul (“output”-ul) acestora;
- În anexe intră lucruri care ocupă mai mult de o pagină ce ar întrerupe firul natural de parcurgere al textului.

A EXTRASE DE COD

...